CLaRa

Continuous Latent Reasoning

Bridging Retrieval and Generation with Advanced RAG Architecture

A Complete Guide: From Concept to Implementation

Apple Research | University of Edinburgh | 2026

TABLE OF CONTENTS

1. The Problem — What is Wrong with Normal RAG?3
2. What is CLaRa?4
3. The Architecture — All Three Components5
4. Stage 1: SCP — Salient Compressor Pretraining6
5. Stage 2: End-to-End Joint Training8
6. The Query Reasoner — The Smart Retriever10
7. Differentiable Top-k Selection11
8. How CLaRa Works at Inference Time12
9. CLaRa vs Normal RAG — Full Comparison13
10. Real Examples with CLaRa14
11. Results from the Paper16
12. Implementation Guide — Step by Step17
13. Frequently Asked Questions22

1. The Problem — What Is Wrong with Normal RAG?

1.1 A Quick Recap of Normal RAG

RAG (Retrieval-Augmented Generation) was designed to fix a core limitation of large language models (LLMs): they can only know what they were trained on. RAG adds an external memory by letting the LLM search a document store before answering.

The standard RAG pipeline works like this:

1. The user sends a question.
2. A retriever searches a document database and finds the most similar documents.
3. Those raw documents are pasted into the LLM's context window (the prompt).
4. The LLM reads the raw text and writes an answer.

Analogy

Imagine a student who must answer exam questions. Instead of memorising everything, they bring textbooks. Before each question, they flip through the books and paste relevant paragraphs onto a piece of paper. Then they read those paragraphs and write the answer. This works — but it has two deep problems.

1.2 Problem 1 — The Retriever and Generator Do Not Talk to Each Other

In normal RAG, the retriever and the answer generator are trained completely separately. The retriever is trained to find documents that look textually similar to the question. The generator is trained to produce good answers from whatever text it receives. Neither one knows what the other needs.

This causes a fundamental mismatch:

- The retriever finds documents based on surface-level word overlap — not based on what would actually help the generator answer correctly.
- If the generator fails because it got the wrong documents, there is no mechanism for that failure to teach the retriever to do better next time.
- The retriever may confidently return a document with many matching words that contains the wrong information — and the system has no way to detect or fix this.

Real Example of Problem 1

Question: "Which city hosted the first modern Olympic Games?"

A normal retriever searches for documents mentioning "Olympics" and finds a document about the Paris 2024 Olympics. It looks relevant — the word Olympics appears many times. But the correct answer is Athens 1896. The generator receives the Paris document and either says "Paris" (wrong) or "I don't know." The retriever was never taught that for this question, you need a document about 1896 specifically.

1.3 Problem 2 — The Generator Reads Too Much

Every time a user asks a question, the generator receives the full raw text of the retrieved documents. A typical document might be 500-2,000 words. Retrieving 5 such documents means the generator must process 2,500-10,000 words of raw text just to answer one question.

This creates three cascading problems:

- Speed: Processing long contexts is computationally expensive and slow for users.
- Cost: Longer inputs mean higher API costs at inference time.
- Quality: LLMs are known to get distracted by irrelevant content in long contexts. The useful information may be buried in noise, and the model's attention is diluted.

The Two Problems Together

Normal RAG suffers from both problems simultaneously: a misaligned retriever that finds suboptimal documents, and a generator that must wade through long raw text even from those suboptimal documents. CLaRa was built specifically to solve both problems with a single unified architecture.

2. What Is CLaRa?

2.1 The Definition

CLaRa stands for Continuous Latent Reasoning. It is a unified framework developed by researchers at Apple and the University of Edinburgh that solves both RAG problems at once by making retrieval and generation operate in the same continuous mathematical space.

> **One-Sentence Definition**
>
> CLaRa compresses every document in your knowledge base into compact numerical "memory tokens" once, trains the retriever and generator jointly so they learn from each other, and at query time retrieves and reads only those compact tokens — not raw text.

2.2 The Index Card Analogy

This analogy makes CLaRa intuitive:

Imagine you are a student preparing for an open-book exam. Instead of bringing 50 full textbooks, you hire a smart research assistant. The assistant reads every book and writes a compact index card for each one — not a summary of every sentence, but only the critical facts needed to answer exam-style questions. The index cards are based on actual exam questions, so they are tuned to what matters.

On exam day, you receive a question. You look through the index cards and find the 3 most relevant ones. You read those 3 cards — not 3 full books — and write your answer. Your performance on the exam also helps the assistant learn to write better cards next time.

Normal RAG	CLaRa
Textbooks	Index cards (memory tokens)
Reads full raw text every question	Reads compact tokens only
Research assistant trained separately	Assistant trained based on exam performance
No feedback from exam results	Exam mistakes improve future cards

2.3 The Three Core Innovations

Innovation 1 — Document Compression into Memory Tokens

Every document is fed through a trained compressor that converts it from raw text into a small set of dense numerical vectors called memory tokens. These vectors capture the meaning and key facts of the document in a compact form. This is done once, offline, before any user ever asks a question.

Innovation 2 — Shared Continuous Representation Space

Both the retriever (which encodes queries) and the generator (which produces answers) operate in the same continuous mathematical space. This means a query vector and a document memory token vector are directly comparable — they use the same coordinate system. In normal RAG, the retriever works in embedding space and the generator works in text space — they are fundamentally mismatched.

Innovation 3 — End-to-End Joint Training

The retriever and generator are trained together using a single loss function: next-token prediction. When the generator makes a mistake (predicts the wrong next word), the error signal flows backward through the generator and then through the document selection process all the way to the retriever. The retriever learns: "To help the generator answer correctly, I need to find documents like these." No explicit retrieval labels are needed.

3. The Architecture — All Three Components

CLaRa has three interconnected components. All three are built on the same base LLM (e.g., Mistral-7B) but use different LoRA (Low-Rank Adaptation) adapter layers to specialize their behaviour.

C Compressor	Takes a raw document and converts it into a small set of memory token vectors. Trained in Stage 1 (SCP). Frozen during Stage 2.
QR Query Reasoner	Takes a user query and encodes it into the same vector space as the memory tokens. Trained during Stage 2 jointly with the generator.
G Generator	Receives the query vector and the top-k selected memory token sets. Produces the final answer. Trained during Stage 2.

Key Insight — Shared Base Model

All three components share the same pretrained LLM weights as their foundation. The Compressor uses one LoRA adapter, the Query Reasoner uses another, and the Generator uses a third. This shared foundation is what makes the continuous representation space work: all three modules speak the same "mathematical language" because they started from the same base.

3.1 How the Components Interact

5. The Compressor processes each document once and stores the resulting memory tokens in a vector index.
6. When a query arrives, the Query Reasoner encodes it into a query vector.
7. The query vector is compared against all stored memory token vectors using cosine similarity.
8. The top-k most similar document memory token sets are selected.
9. The Generator receives the query vector concatenated with the top-k memory token sets and produces the answer.

4. Stage 1: SCP — Salient Compressor Pretraining

4.1 What SCP Is and Why It Exists

Before CLaRa can do anything, it needs a compressor that creates high-quality memory tokens. If the memory tokens are poor — if they miss important facts or are semantically vague — then the retrieval and generation that follow will also be poor. SCP (Salient Compressor Pretraining) is the first training stage that teaches the compressor to create semantically rich, compact representations.

Previous compression methods trained models to reconstruct the original document token by token. This is called reconstruction loss and it has a critical flaw: the model wastes its limited capacity learning to reproduce every word, including filler words, punctuation, and irrelevant sentences. The resulting compressed representation is faithful to the surface form but misses the semantic core.

What SCP Fixes

SCP teaches the compressor to retain only the semantically salient information — the facts, relationships, and concepts that are necessary to answer real questions about the document. It does this by building a synthetic training dataset of QA pairs and paraphrases that explicitly expose what information matters.

4.2 How the SCP Training Data Is Built

SCP works by generating synthetic training data from a large corpus of documents (the paper uses 2 million Wikipedia articles). For each document, three types of supervision signals are created:

Signal 1: Simple QA Pairs

An LLM (the paper uses Qwen-32B) generates simple question-answer pairs from the document. Each pair targets one atomic fact. For example, from a document about Aristotle: Q: "Where was Aristotle born?" A: "Stagira, Greece." The constraint is that each new question must cover a fact not covered by previous questions, ensuring comprehensive coverage of the document's content.

Why this matters: The compressor must produce memory tokens from which these atomic facts can be recovered. This forces the compressor to retain specific details, not just vague summaries.

Signal 2: Complex QA Pairs

These are multi-hop questions that require combining two or more facts from across the document. For example: "What political philosophy influenced the city that Aristotle tutored Alexander the Great in?" The LLM is guided to create questions that connect previously unlinked pieces of information.

Why this matters: Complex QA forces the compressor to retain relational information and conceptual structure — not just individual facts but how facts connect to each other.

Signal 3: Paraphrases

The original document is reworded in multiple ways: different sentence structure, different vocabulary, same meaning. The compressor must produce similar memory tokens regardless of how the document is worded.

Why this matters: This forces the compressor to learn meaning rather than surface pattern. A document about "dogs" and a paraphrase about "canines" should produce the same memory tokens. This is what makes the representation truly semantic.

4.3 Verification and Regeneration

Each generated QA pair and paraphrase is automatically verified by the LLM for factual consistency and information coverage. If the generated outputs miss important facts from the document, the LLM generates additional QA pairs to fill the gaps. This iterative process runs for up to 10 rounds. Samples that still fail coverage criteria after 10 rounds are discarded.

This quality control ensures the compressor only learns from fully accurate, comprehensive training examples.

4.4 How the Compressor Is Trained

The compressor is built on a shared base LLM with a specialized LoRA adapter. For a document with tokens t1, t2, ..., tm, the compressor appends l learnable memory tokens m1, m2, ..., ml and runs the forward pass. The final hidden states of the memory tokens become the compressed representation Mi.

Two loss functions are used together:

Cross-Entropy Loss (CE)

The generator is trained to produce correct answers from the compressed memory tokens. This loss measures how well the memory tokens preserve the information needed for generation.

Compression Alignment Loss (MSE)

The mean squared error between the average hidden state of the original document tokens and the average hidden state of the memory tokens. This ensures the memory tokens remain semantically aligned with the original document — they cannot drift into a completely different mathematical region.

Total Training Loss

L_total = L_CE + lambda * L_MSE

where lambda balances generative quality and semantic alignment. A higher lambda keeps the memory tokens closer to the original document's representation. A lower lambda allows more compression flexibility.

Compression Ratio

The compression ratio CR is defined as: CR = (number of tokens in original document) / (number of memory tokens). A CR of 16 means a 512-token document is compressed into 32 memory tokens. CLaRa is evaluated at CR = 4, 16, and 128. Even at CR=16, CLaRa outperforms systems reading the full raw text.

5. Stage 2: End-to-End Joint Training

5.1 The Core Goal of Stage 2

After SCP produces a strong compressor, CLaRa proceeds to Stage 2: training the query reasoner and generator jointly in an end-to-end fashion. This is where the magic of CLaRa comes from — retrieval and generation are no longer separate processes but a single unified system optimized together.

Why Joint Training Changes Everything

In normal RAG, if the generator fails to answer correctly because it received the wrong documents, the retriever never learns from this failure. In CLaRa, the generator's failure becomes a training signal that directly updates the retriever. Over thousands of training examples, the retriever learns: "When a question has property X, I need to find documents with property Y." This alignment happens automatically without any human-labeled retrieval data.

5.2 The Training Procedure

The training uses question-answer pairs where the reference answer is known. For each training example:

10. The query q is passed through the Query Reasoner (theta_qr), producing a query embedding q.
11. The frozen compressor has already produced memory token embeddings Mi for each candidate document.
12. Cosine similarity between q and each Mi gives a relevance score si.
13. Differentiable top-k selection chooses the k highest-scoring documents.
14. The selected memory tokens M(1:k) are concatenated with q and fed to the generator.
15. The generator produces the answer token by token.
16. The cross-entropy loss L_CLaRa measures how far the generated answer is from the correct answer.
17. Gradients flow backward through the generator, through the top-k selection, and into the query reasoner weights. The compressor weights remain frozen.

5.3 The CLaRa Training Loss

CLaRa Loss Function

L_CLaRa(theta_q, theta_g) = - sum over (q, M(1:k), a*) of: sum(t=1 to |a*|) log p_theta_g(a*_t | q, M(1:k), a*_<t)

In plain English: the loss measures how well the generator predicts each word of the correct answer, given the query and the selected memory tokens. Lower loss = better answers. The gradient of this loss updates both theta_q (query reasoner) and theta_g (generator).

5.4 Instruction Tuning (Optional Stage 2b)

After the main joint training, an optional instruction tuning step can be performed. This uses downstream task-specific training data (e.g., QA datasets with specific answer formats) to further align the compressor and generator to the target use case.

The paper shows that instruction-tuned CLaRa consistently outperforms pretraining-initialized CLaRa, especially on harder multi-hop reasoning tasks.

5.5 Why the Compressor Must Be Frozen

A critical design choice in Stage 2 is that the compressor weights are frozen — they do not update. The memory tokens for all documents are pre-computed and stored. This is necessary because:

- Efficiency: Re-encoding all documents every training step would be prohibitively expensive for large knowledge bases.
- Stability: If the compressor were updating, the stored memory tokens would become stale (computed with old weights) and the retrieval system would break.
- Division of responsibilities: SCP trains the compressor to produce semantically rich representations. Stage 2 trains the query reasoner to find those representations and the generator to use them.

6. The Query Reasoner — The Smart Retriever

6.1 What Makes the Query Reasoner Different

A standard dense retriever encodes the query as a vector by looking at the words in the question and finding their semantic representation. The query vector captures what the question is literally asking about.

The CLaRa Query Reasoner does something fundamentally different: it is trained to produce a query vector that anticipates what information is needed to generate a correct answer — even if that information involves concepts not explicitly mentioned in the question.

The Multi-Hop Example from the Paper

Question: "How many yards did the nephew of Ivory Lee Brown get during his 2004 true freshman season?"

A standard retriever would search for: Ivory Lee Brown, nephew, yards, 2004, freshman season.

The CLaRa Query Reasoner, after end-to-end training, internally generates reasoning tokens including: NFL, Oklahoma, freshman record, Adrian Peterson.

These words do NOT appear in the question. But the query reasoner has learned that to answer this type of question, you need a document that connects these concepts. The retrieved document about Adrian Peterson mentions that his uncle is Ivory Lee Brown and that he set the NCAA freshman rushing record with 1,925 yards. The answer is 1,925 yards.

A standard retriever would never find this document because it focuses on the wrong keywords.

6.2 How This Capability Is Learned

The query reasoner's ability to anticipate reasoning-relevant concepts emerges naturally from end-to-end training. It is not explicitly programmed. Here is the learning mechanism:

18. During training, the query reasoner produces a query vector q.
19. Documents are ranked by cosine similarity to q.
20. The top-k selected documents are used by the generator to produce an answer.
21. If the answer is wrong, the loss is high.
22. The gradient signal tells the query reasoner: "The documents you selected didn't help. Your query vector q pointed in the wrong direction in the vector space."
23. Over thousands of such corrections, the query reasoner learns to produce query vectors that consistently point toward the memory token regions that help generate correct answers.

Technical Details

The query reasoner uses the same base LLM as the compressor, initialized from the SCP-trained compressor weights (so it starts from a strong semantic foundation). It has its own LoRA adapter. It processes the query tokens q1, ..., qv and produces an output embedding q that has the same dimensionality as the document memory tokens Mi. The similarity score between query and document i is: $s_i = \cos(q, M_i)$.

7. Differentiable Top-k Selection

7.1 The Gradient Problem

End-to-end training requires gradients to flow from the generator all the way back to the query reasoner. But between them sits the top-k selection operation: choosing the k documents with the highest similarity scores. This is a discrete operation — document i is either selected (1) or not (0). Discrete decisions have zero gradients almost everywhere, which means standard backpropagation cannot flow through them.

This is known as the broken gradient problem, and it is why previous approaches to end-to-end RAG training used slow, unstable reinforcement learning methods instead.

7.2 The Straight-Through (ST) Estimator Solution

CLaRa solves this with a technique called Straight-Through estimation, which makes the top-k selection behave differently during the forward pass (when computing outputs) and the backward pass (when computing gradients):

Forward pass (inference behaviour)

Hard selection: documents are either selected (1) or not (0). The top-k documents are determined by their similarity scores. This is discrete — the output is a binary selection matrix. This ensures the actual retrieval behaviour is sharp and deterministic.

Backward pass (learning behaviour)

Soft selection: instead of the hard 0/1 decisions, the gradient is computed as if the selection were a softmax-weighted combination of all documents. The softmax temperature controls how "soft" this is. A lower temperature approximates hard selection more closely.

The Straight-Through trick: in the forward pass, use hard selection. In the backward pass, use the gradient of the soft selection as if the forward pass had also been soft. This is mathematically inconsistent but empirically works extremely well — it provides stable, meaningful gradient signals while preserving sharp retrieval behaviour.

Why This Is Better Than RL

Previous methods (like DDR-RAG) used reinforcement learning to train retrieval: sample many different document sets, run the generator on each, observe which ones produced better answers, and use that as a reward signal. This requires hundreds of generator calls per training step, is computationally expensive, and suffers from high variance. The ST estimator provides a gradient signal in a single forward-backward pass, making training 10-100x more efficient.

8. How CLaRa Works at Inference Time

8.1 Offline Phase (One-Time Setup)

Before any user query, every document in the knowledge base is processed:

24. Each document is passed through the frozen, trained compressor.
25. The compressor produces l memory tokens per document (l is a hyperparameter, e.g., 32).
26. These memory token vectors are stored in a vector index (e.g., FAISS).

This is done once and the index is kept until documents change. Adding a new document means just running the compressor on it and adding the resulting vectors to the index.

8.2 Online Phase (Per-Query)

When a user sends a question:

27. The query text is passed to the Query Reasoner, which produces a query vector q in the same space as all document memory tokens.
28. Cosine similarity between q and all stored document memory vectors is computed. The top-k documents are selected (default k=5 in experiments).
29. The query vector q and the memory tokens of the top-k documents are concatenated to form the generator's input.
30. The Generator produces the answer autoregressively, token by token.

Context Length Comparison

Normal RAG at CR=1: 5 documents x 512 tokens each = 2,560 tokens fed to the generator.

CLaRa at CR=16: 5 documents x 32 memory tokens each = 160 tokens fed to the generator.

CLaRa at CR=16 feeds the generator 16 times less input — yet achieves higher accuracy.

9. CLaRa vs Normal RAG — Full Comparison

Feature	Normal RAG	CLaRa
What generator reads	Full raw text of documents	Compact memory token vectors
Retriever training signal	Surface word similarity only	Generator's answer quality (backprop)
Training approach	Retriever and generator separate	Joint end-to-end training
Feedback loop	None — retriever never learns from generation errors	Full — generation errors update retriever
Representation space	Two different spaces (embedding + text)	One shared continuous space
Context length	Full document length per doc	l memory tokens per doc (e.g. 32)
Document encoding	Every query (re-encoded)	Once offline, then reused
Retrieval labels needed	Often required (human annotations)	Not needed — generation loss suffices
Retriever training method	Contrastive learning with labeled pairs	Weak supervision from NTP loss
Top-k selection gradient	Not differentiable (RL required)	Differentiable (ST estimator)
Query intelligence	Encodes literal question words	Anticipates reasoning-relevant concepts
Multi-hop reasoning	Often fails (wrong docs retrieved)	Strong (query reasoner bridges hops)

10. Real Examples with CLaRa

10.1 Single-Hop Question (Simple Factual)

Question

"What is the capital of France?"

Normal RAG process

1. Retriever searches for documents containing 'capital' and 'France'.
2. Finds a Wikipedia article on France. Full article: 8,000 tokens.
3. Generator reads 8,000 tokens and answers 'Paris'.

Correct — but inefficient. 8,000 tokens processed for a 1-word answer.

CLaRa process

1. France article was compressed offline into 32 memory tokens. These tokens encode: capital=Paris, population, history, government, etc.
2. Query Reasoner encodes 'What is the capital of France?' into a query vector.
3. The France memory tokens rank highest. Only 32 tokens retrieved.
4. Generator reads 32 tokens and answers 'Paris'.

Correct — and 250x fewer tokens processed.

10.2 Multi-Hop Question (Complex Reasoning)

Question

"The author of 'Pride and Prejudice' was from which country, and what language was that country's national anthem originally written in?"

Normal RAG process

1. Retriever searches for 'Pride and Prejudice author country national anthem language'.
2. Finds a document about Pride and Prejudice. Does not retrieve a document about the UK national anthem.
3. Generator has incomplete information and either hallucinate or says 'I don't know'.

Fails — the retriever focused on surface keywords and missed the second hop.

CLaRa process

1. Query Reasoner encodes the question and internally adds context: Jane Austen → England → UK → national anthem → 'God Save the King' → originally English.
2. Retrieves memory tokens from: (a) Jane Austen article (b) United Kingdom article.
3. Generator reads the two compact memory token sets.
4. Produces: 'Jane Austen was British. The UK national anthem God Save the King was originally written in English.'

Correct — the query reasoner bridged the two hops.

10.3 Question That Is NOT in the Knowledge Base

Question

"What is the latest iPhone model?" (asked in 2030, knowledge base only has 2023 data)

Both Normal RAG and CLaRa

Both systems will retrieve whatever iPhone-related documents they have and generate an answer based on stale information. CLaRa does not solve the knowledge cutoff problem — it solves the retrieval-generation alignment problem. For out-of-date information, you need to either update the knowledge base or add a web search tool.

11. Results from the Paper

11.1 Benchmarks Used

CLaRa was evaluated on four QA benchmarks of increasing difficulty:

- NQ (Natural Questions): Single-hop questions based on real Google search queries. Answers come from Wikipedia.
- HotpotQA: Multi-hop questions requiring reasoning across two Wikipedia articles.
- MuSiQue: Multi-hop questions requiring 2-4 reasoning steps.
- 2WikiMultihopQA (2Wiki): Questions requiring reasoning across two Wikipedia articles with structured relation chains.

11.2 Key Findings

Finding 1 — Compression beats raw text

At CR=16 (reading 16x less text), CLaRa-Mistral-7B achieves +2.36% better F1 on NQ compared to systems reading the full raw uncompressed text. This means compressed representations can actually filter noise and help the generator focus on what matters.

Finding 2 — Beats all supervised rerankers

CLaRa at CR=4 outperforms supervised rerankers like RankZephyr-7B that are specifically trained with human-labeled relevance data. CLaRa achieves this with zero retrieval labels — it learns from generation quality alone.

Finding 3 — Joint training adds significant value

CLaRa-Mistral-7B improves F1 from 51.01 (pretraining only) to 51.41 (joint training) on NQ. On 2Wiki: 43.65 to 47.18. On multi-hop tasks, the improvement is larger because the query reasoner's ability to bridge hops becomes especially valuable.

Finding 4 — Oracle performance shows headroom

In Oracle settings (where the correct document is guaranteed to be in the top-20 pool), F1 exceeds 75% on both NQ and HotpotQA. This shows that when retrieval is perfect, CLaRa's generation is highly accurate. The remaining gap in Normal settings is due to retrieval difficulty, not generation quality.

12. Implementation Guide — Step by Step

12.1 Prerequisites

Before you begin, ensure you have the following:

- Python 3.10 or later
- PyTorch 2.0+ with CUDA support (a GPU with at least 40GB VRAM for Mistral-7B)
- The official CLaRa code from: https://github.com/apple/ml-clara
- A base LLM (Mistral-7B or Phi-4-mini as used in the paper)
- FAISS for vector indexing
- HuggingFace Transformers and PEFT (for LoRA)

12.2 Step 1 — Environment Setup

```
# Clone the repository
git clone https://github.com/apple/ml-clara
cd ml-clara

# Create virtual environment
python -m venv clara_env
source clara_env/bin/activate  # Linux/Mac

# Install dependencies
pip install -r requirements.txt

# Verify GPU is available
python -c "import torch; print(torch.cuda.is_available())"
```

12.3 Step 2 — Prepare Your Documents

Your documents need to be in a simple format: one document per line, plain text. Each line is one document. The document ID is its line number.

```
# documents.txt format (one document per line):
Aristotle was an ancient Greek philosopher born in 384 BC in Stagira...
The capital of France is Paris, which has been the nation's capital since...
Photosynthesis is the process by which plants convert sunlight into glucose...
```

Then tokenize and chunk your documents to the target length (the paper uses chunks of 100 tokens):

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('mistralai/Mistral-7B-v0.1')
```

```
def chunk_document(text, max_tokens=100):
    tokens = tokenizer.encode(text, add_special_tokens=False)
    chunks = []
    for i in range(0, len(tokens), max_tokens):
        chunk = tokens[i:i + max_tokens]
        chunks.append(tokenizer.decode(chunk))
    return chunks

all_chunks = []
with open('documents.txt') as f:
    for line in f:
        all_chunks.extend(chunk_document(line.strip()))
```

12.4 Step 3 — Train the Compressor with SCP

This is Stage 1. You need the synthetic QA training data first, then train the compressor.

Step 3a — Generate SCP Training Data

```
# The paper provides scripts to generate SCP data
# This uses a large LLM to generate QA pairs from your documents

python scripts/generate_scp_data.py \
    --input_docs documents.txt \
    --output_dir scp_data/ \
    --model_name Qwen/Qwen2.5-32B-Instruct \
    --num_simple_qa 5 \
    --num_complex_qa 3 \
    --num_paraphrases 2 \
    --max_verification_rounds 10
```

Step 3b — Train the Compressor

```
python train_compressor.py \
    --base_model mistralai/Mistral-7B-v0.1 \
    --scp_data_dir scp_data/ \
    --num_memory_tokens 32 \
    --compression_ratio 16 \
    --lora_rank 16 \
    --lora_alpha 32 \
    --learning_rate 2e-4 \
    --num_epochs 3 \
    --batch_size 8 \
    --lambda_mse 0.1 \
    --output_dir checkpoints/compressor/
```

> **What num_memory_tokens controls**
>
> num_memory_tokens = l = the number of memory tokens per document chunk.
> compression_ratio = CR = (original_tokens) / (memory_tokens).
> If your chunks are 512 tokens and you want CR=16, set num_memory_tokens=32.

> Fewer memory tokens = more compression = faster retrieval and generation, but potentially lower accuracy at extreme ratios (CR=128+).

12.5 Step 4 — Offline Document Compression

With the trained compressor, compress your entire knowledge base. This is done once.

```
python compress_documents.py \
    --compressor_checkpoint checkpoints/compressor/ \
    --input_chunks chunks.json \
    --output_dir compressed_index/ \
    --batch_size 32 \
    --device cuda

# Output: compressed_index/
#   memory_tokens.pt     -- tensor of shape [N_docs, l, hidden_dim]
#   doc_ids.json         -- mapping from index position to document ID
#   faiss.index          -- FAISS index for fast similarity search
```

12.6 Step 5 — Train the Query Reasoner and Generator (Stage 2)

Stage 2 jointly trains the query reasoner and generator using your downstream QA data.

```
python train_joint.py \
    --base_model mistralai/Mistral-7B-v0.1 \
    --compressor_checkpoint checkpoints/compressor/ \
    --compressed_index compressed_index/ \
    --qa_data_dir qa_train_data/ \
    --top_k 5 \
    --st_temperature 0.5 \
    --lora_rank 16 \
    --learning_rate 1e-4 \
    --num_epochs 3 \
    --batch_size 4 \
    --gradient_accumulation_steps 8 \
    --output_dir checkpoints/joint/
```

> **What st_temperature controls**
>
> The Straight-Through temperature controls how 'soft' the document selection is during training gradients. A value of 0.5 (as used in the paper) provides a reasonable balance between gradient smoothness and sharp selection approximation. Lower values = sharper, closer to hard selection. Higher values = smoother gradients but less accurate approximation.

12.7 Step 6 — Run Inference

With all components trained, you can now answer queries:

```
from clara import CLaRaInference

# Load the full trained system
clara = CLaRaInference(
    compressor_checkpoint='checkpoints/compressor/',
    joint_checkpoint='checkpoints/joint/',
    compressed_index_dir='compressed_index/',
    top_k=5,
    device='cuda'
)

# Ask a question
query = "Which city hosted the first modern Olympic Games?"
answer = clara.answer(query)
print(answer)  # --> Athens, Greece (1896)

# Get more details including which documents were retrieved
result = clara.answer_with_details(query)
print('Answer:', result['answer'])
print('Retrieved documents:', result['retrieved_doc_ids'])
print('Similarity scores:', result['scores'])
```

12.8 Step 7 — Adding New Documents

CLaRa handles new documents gracefully because encoding is independent of training:

```
# Add new documents after initial setup
new_docs = ['The Eiffel Tower was built in 1889 by Gustave Eiffel...']

clara.add_documents(
    new_docs,
    doc_ids=['eiffel_001']  # optional custom IDs
)

# The system immediately uses the new documents for retrieval
# No retraining needed!
answer = clara.answer('Who built the Eiffel Tower?')
print(answer)  # --> Gustave Eiffel
```

Important: No Retraining for New Documents

Because the compressor is frozen, new documents can be added at any time without retraining. The compressor encodes the new document into memory tokens and they are added to the FAISS index. This is one of CLaRa's most practical advantages over approaches that require full system retraining when the knowledge base changes.

12.9 Recommended Hyperparameters (from the paper)

Parameter	Value	Notes
Base model	`Mistral-7B-v0.1`	Also works with Phi-4-mini for lighter setup

Memory tokens (l)	32	At CR=16 for 512-token chunks
Compression ratio	16x	Best accuracy/speed tradeoff
Top-k documents	5	Retrieved per query at inference
LoRA rank	16	For all three LoRA adapters
SCP learning rate	2e-4	For compressor training
Joint learning rate	1e-4	For query reasoner + generator
ST temperature	0.5	For differentiable top-k
Lambda MSE	0.1	Alignment loss weight in SCP
SCP epochs	3	Over the 2M Wikipedia corpus
Joint epochs	3	Over downstream QA data

13. Frequently Asked Questions

Q1: Do I need to retrain CLaRa when I add new documents?

Answer

No. New documents only need to pass through the frozen compressor, which produces their memory tokens in seconds. These tokens are added to the FAISS index and immediately available for retrieval. The query reasoner and generator do not need to be retrained. You only need to retrain if you change the underlying LLM, the compression ratio, or the knowledge base changes dramatically in domain.

Q2: How does CLaRa handle very long documents?

Answer

CLaRa chunks long documents into smaller pieces (e.g., 100-512 tokens each) and compresses each chunk independently. Each chunk gets its own set of memory tokens. At retrieval time, individual chunks (not whole documents) are retrieved. This actually works better than retrieving whole documents in many cases.

Q3: What happens if the compressor loses important information during compression?

Answer

This is a real risk at extreme compression ratios (CR=128+). The SCP training mitigates this by explicitly training the compressor to preserve question-answering-relevant facts. At CR=16, the paper shows no meaningful accuracy loss compared to reading full text. At CR=128, performance drops slightly but CLaRa still outperforms many baselines. If accuracy is critical, use CR=4 or 8.

Q4: Can CLaRa be used with any LLM?

Answer

CLaRa is LLM-agnostic in principle — it uses LoRA adapters on top of a base LLM, so it can work with any LLM that supports LoRA. The paper tests Mistral-7B-v0.1 and Phi-4-mini. Larger base models will generally produce better compressed representations and better generation. The key constraint is that all three components (compressor, query reasoner, generator) must use the same base LLM.

Q5: How is CLaRa different from just making a very small model?

Answer

CLaRa is not training a small model — it is training a compression function that converts documents into compact numerical representations that a full-sized LLM can still reason over. The generator is still a full 7B or larger model. CLaRa reduces the input length to the generator, not the model size. This is fundamentally different from model distillation or quantization.

Q6: What is the computational cost compared to normal RAG?

Answer

Offline: CLaRa requires an initial compression pass (running all documents through the compressor), which is a one-time cost. Normal RAG has no offline cost beyond building a standard embedding index.

Online (per query): CLaRa's generator processes 16x fewer tokens (at CR=16), making generation significantly faster. The query reasoner is the same speed as a normal dense retriever.

Training: CLaRa requires two training stages (SCP + joint), which is more expensive than training a normal RAG retriever. However, training is a one-time cost and the improved accuracy is sustained at inference.

Q7: Is CLaRa suitable for production use?

Answer

CLaRa is a research system (February 2026 preprint). The code is available at https://github.com/apple/ml-clara. For production use, you would want to: (1) evaluate on your specific domain data, (2) set up proper monitoring for retrieval quality, (3) handle edge cases like very short documents, (4) consider serving infrastructure for the three separate model components. The system is more complex to deploy than a standard RAG pipeline but offers significantly better performance.

SUMMARY — WHAT YOU NOW KNOW ABOUT CLaRa

The Problem CLaRa Solves

Normal RAG has two problems: (1) the retriever and generator are disconnected — the retriever learns from word similarity, not from whether the generator can answer correctly; (2) the generator reads full raw text which is slow and noisy.

The Solution

CLaRa compresses every document into compact memory tokens once. It trains the retriever (query reasoner) and generator jointly so that generation mistakes directly update the retriever. At inference, only memory tokens are fed to the generator — not raw text.

The Three Key Technologies

SCP (Salient Compressor Pretraining): teaches the compressor to retain semantically important facts using QA pairs and paraphrases as training signals.

Joint End-to-End Training: trains retriever and generator together using a single cross-entropy loss, allowing gradient flow from generation back to retrieval.

Differentiable Top-k via ST Estimator: makes document selection differentiable so gradients can flow through it without requiring reinforcement learning.

The Result

State-of-the-art performance on NQ, HotpotQA, MuSiQue, and 2Wiki benchmarks. At 16x compression (reading 16x less text than normal RAG), CLaRa achieves higher accuracy. No retrieval labels needed. New documents can be added without retraining.

CLaRa: Bridging Retrieval and Generation with Continuous Latent Reasoning
Jie He, Richard He Bai, Sinead Williamson, Jeff Z. Pan, Navdeep Jaitly, Yizhe Zhang
Apple Research & University of Edinburgh | arXiv:2511.18659v3 | February 2026
Code: https://github.com/apple/ml-clara